

TOP 50 CÂU HỎI PHỎNG VẤN BACKEND

(2026) Phiên bản Học Trực Quan kèm Đáp Án

System Design | LLD | API | Database | Caching | Message Queues | Microservices

★ Chinh phục phỏng vấn Backend tại FAANG

Lộ trình phỏng vấn Backend - Luồng hoạt động:



💡 Vì Sao Chọn Cẩm Nang Này?:

- 50 câu hỏi được hỏi nhiều nhất tại FAANG | FAANGM | Amazon | Flipkart | Google năm 2026
- Đáp án trực quan kèm sơ đồ - không chỉ lý thuyết suông
- Bao quát HLD • LLD • API • DB • System Design
- Mỗi đáp án đều có: Lý do (Why) + Sơ đồ (Diagram) + Ví dụ thực tế (Real Example)

📖 Trong cẩm nang này bạn sẽ học:

Danh mục:

- 1-10 • System Design (Rate Limiting, Caching, Load Balancer, v.v.)
- 11-20 • Database & SQL / NoSQL
- 21-30 • API Design & Auth • JWT • OAuth
- 31-40 • Microservices & Message Queues + Kafka
- 41-50 • LLD & OOP & Thực tế + Payments + Thiết kế Instagram

Tổng cộng: 50 câu hỏi & đáp án - trực quan, thực tế, sẵn sàng phỏng vấn

💬 Chinh phục 50 câu hỏi phỏng vấn Backend quan trọng nhất 2026!

Swipe to Page 02 →



Q1-Q4

NGUYÊN LÝ CƠ BẢN SYSTEM DESIGN

🕒 Q1 Rate Limiting là gì?

Đáp án: Kiểm soát số lượng request client được gửi trong một khoảng thời gian.



tokens removed
when request
Refill over time

Token Bucket

🕒 Fixed Window
100 req / 1min window
→ reset

- Vì sao cần: ngăn DDoS • bảo vệ backend • kiểm soát chi phí
- Dùng ở đâu: API Gateway • Auth Service 🔒

📁 Q2 Cache-Aside vs Write-Through vs Write-Behind?

Cache-Aside

App → check → **Cache** → nếu miss → **DB**

App kiểm tra cache trước, nếu miss thì truy vấn DB. Đơn giản, phổ biến nhất

Write-Through: App → ghi vào Cache + DB cùng lúc

Write-Behind: App → ghi vào Cache → ghi bất đồng bộ vào DB sau

Dùng Redis / Memcached cho cache

📁 Load Balancer Algorithms - Thuật toán cân bằng tải?

- **Round Robin:** request → Server1 → Server2 → Server3 xoay vòng
- **Least Connections:** chuyển đến server có ít kết nối đang hoạt động nhất
- **IP Hash:** IP client → hash → luôn về cùng một server
- **Weighted:** trọng số cao hơn → nhận nhiều request hơn

Load Balancer → **Srv A** → **Srv B** → **Srv C**

⚠️ Q4 CAP Theorem là gì?

Consistency • Availability • Partition Tolerance — chỉ chọn 2

- AP = Available + Partition Tolerant → Dynamo, Cassandra
- CP = Consistent + Partition Tolerant → HBase, MongoDB ✓
- CA = Consistent + Available → chỉ với single node, không phải hệ phân tán

Trong hệ phân tán, P là điều bắt buộc → phải chọn AP hoặc CP

💡 Vì sao nhà tuyển dụng hỏi câu này:

- Kiểm tra tư duy về khả năng mở rộng thực tế
- Cho thấy bạn hiểu đánh đổi (trade-off), không chỉ định nghĩa suông
- Hệ thống Backend = thiết kế để chịu lỗi

📌 Ví dụ thực tế:

- API Gateway rate limiting → 1000 req/phút mỗi user
- Cache-Aside → cache newsfeed của Instagram
- Load Balancer → AWS ELB dùng Round Robin
- CAP → Cassandra = AP, MongoDB = CP

💬 Giờ bạn đã hiểu nền tảng - 4 mẫu thiết kế cốt lõi của System Design!

Swipe to Page 03 →

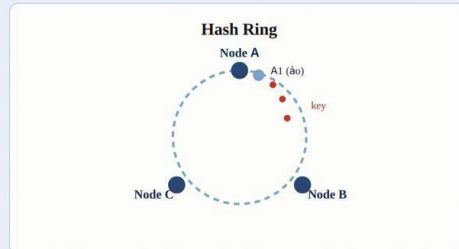


Q5-Q8

SHARDING, DB CƠ BẢN

Q5 Consistent Hashing là gì?

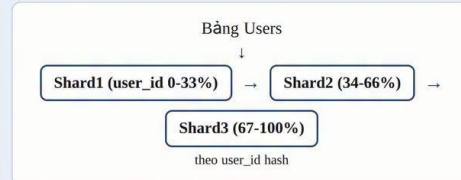
Đáp án: Vòng hash (hash ring) với các virtual node, tránh phải hash lại toàn bộ khi thêm/bớt server.



- Vì sao cần: mở rộng không downtime, giảm thiểu di chuyển key
- Dùng ở đâu: DynamoDB, Cassandra, CDN, Load Balancer

Q6 Sharding vs Partitioning?

Đáp án: Sharding = chia ngang dữ liệu qua các DB khác nhau (khác máy), Partitioning = chia trong cùng một DB (cùng instance).



- Ưu điểm: tăng khả năng ghi • đọc
- Nhược điểm: join phức tạp, resharding khó
- Vì sao cần: mở rộng ngang khi dữ liệu tăng
- Dùng ở đâu: Ứng dụng quy mô lớn, microservices

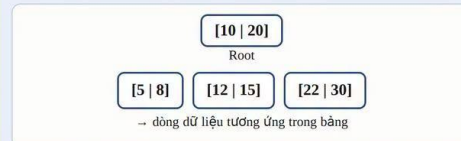
Q7 Khi nào dùng SQL vs NoSQL?

SQL	NoSQL
ACID, tính nhất quán mạnh	BASE, nhất quán cuối cùng (eventual)
Có cấu trúc, hỗ trợ join	Schema linh hoạt, mở rộng ngang
Scale theo chiều dọc	Tự động sharding

- Dùng ở đâu: SQL → Payments, banking, quản lý kho
- Dùng ở đâu: NoSQL → Feed, chat, IoT, session store
- Vì sao cần: cân bằng giữa nhất quán, khả năng mở rộng và linh hoạt

Q8 Indexing & DB Index hoạt động ra sao?

Đáp án: B-Tree index $O(\log n)$ so với quét toàn bộ (full scan) $O(n)$.



Loại: Primary (clustered) → Secondary + Composite (col1, col2)

- Vì sao nhanh: tránh quét toàn bộ, tra cứu nhanh
- Khi nào chậm: ghi dữ liệu cần cập nhật index

Vì sao nhà tuyển dụng hỏi câu này:

- Kiểm tra tư duy về tăng trưởng dữ liệu & khả năng mở rộng
- Hiểu đánh đổi giữa sharding và hiệu năng
- Lựa chọn DB ảnh hưởng đến tính nhất quán & khả dụng

Ví dụ thực tế:

- Consistent Hashing → vòng AWS Dynamo, Cassandra
- Discord sharding 10M+ theo server_id
- SQL: Stripe dùng Postgres cho thanh toán
- NoSQL: newsfeed Instagram = Cassandra
- Index: Uber index truy vấn địa lý = nhanh hơn 100 lần

🗨️ Giờ bạn đã biết sharding, hashing & DB cốt lõi - 4 mẫu phỏng vấn DB bắt buộc phải biết!

Swipe to Page 04 →



Q9-Q12 ĐÀO SÂU VỀ DATABASE

Q9 Vấn đề N+1 Query là gì? 📖 !

Đáp án: Lặp truy vấn DB nhiều lần trong vòng lặp gây ra rất nhiều query.

App → 10x → DB → 10 query

Cách fix: SELECT * WHERE id IN (...)

Giải quyết bằng JOIN / Batch / Eager loading.

- Vì sao cần: ngăn "kẻ giết hiệu năng" • giảm độ trễ API
- Dùng ở đâu: ORM như Hibernate, Django

Q10 Normalization vs Denormalization?

Đáp án: Normalization giảm dư thừa dữ liệu 1NF 2NF 3NF, Denormalization thêm dữ liệu dư thừa để đọc nhanh hơn.

Normalization

- Giảm dư thừa
- 1NF / 2NF / 3NF
- Ít trùng lặp

Denormalization

- Thêm dư thừa
- Đọc nhanh hơn
- Nhiều trùng lặp hơn

Vì sao cần: giảm bất thường (anomaly) đổi lấy tốc độ.

Dùng ở đâu: OLTP dùng normalized, OLAP dùng denormalized.

Q11 ACID vs BASE?

Đáp án: ACID = Atomic Consistent Isolated Durable (SQL), BASE = Basically Available Soft state Eventual consistency (NoSQL).

✓ ACID

- Nhất quán mạnh
- Giao dịch nguyên tử
- Cơ sở dữ liệu SQL

☹ BASE

- Khả dụng cao
- Nhất quán cuối cùng
- Cơ sở dữ liệu NoSQL

Vì sao cần: đánh đổi giữa nhất quán mạnh và khả dụng.

Q12 Replication vs Partitioning?

Đáp án: Replication sao chép cùng dữ liệu tới nhiều node để tăng khả dụng, Partitioning chia nhỏ dữ liệu (sharding).

Loại: Master-Slave • Master-Master cho replication.

Replication

DB → DB1 DB2 DB3
bản sao đã nhân bản

Partitioning

Shard A | Shard B | Shard C

Dùng: MySQL replica, Mongo sharding.

💡 Vì sao nhà tuyển dụng hỏi câu này:

- Kiểm tra tư duy tối ưu DB
- Cho thấy bạn hiểu đánh đổi SQL vs NoSQL
- Scaling = chiến lược dữ liệu

Ví dụ thực tế:

- N+1 → newsfeed Instagram 10 query, fix bằng JOIN
- Normalization → thiết kế DB của Flipkart
- ACID → Payments cần ACID, BASE → Cassandra AP
- Replication → MySQL master-slave để scale đọc

🗨️ Giờ bạn đã hiểu nội tại DB - cốt lõi cho phỏng vấn Backend!

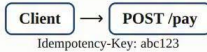
→ Swipe to Page 05



Q13-Q16 THIẾT KẾ API

🕒 Q13 Idempotency là gì?

Đáp án: Gửi cùng một request nhiều lần = kết quả giống nhau.



Server kiểm tra Key Store (Redis)

Tồn tại? → trả kết quả đã lưu. Không → xử lý & lưu kết quả
✓ → không bị tính phí trùng ✓

- Dùng ở đâu: Payment APIs - Stripe, UPI, Razorpay

📖 Q14 Các loại Pagination? Offset vs Cursor vs Keyset

- Offset: `?offset=20&limit=10` → bỏ qua 20 dòng - chậm $O(n)$, mất dữ liệu khi có insert mới
- Cursor: `?cursor=eyJ...&limit=10` → `next_cursor` - nhanh $O(1)$, ổn định cho cuộn vô hạn
- Dùng offset cho trang quản trị
- Dùng cursor cho feed

	Offset	Cursor
Ưu điểm	Đơn giản / dễ	Nhanh / ổn định vô hạn
Nhược điểm	Chậm $O(n)$ / lệch khi có data mới	Cần lưu trạng thái cursor, khó cài đặt hơn
Phù hợp	Dashboard quản trị / ít dữ liệu	Feed / dữ liệu lớn

◦ Q15 REST vs GraphQL vs gRPC?

REST	GraphQL	gRPC
Theo resource, query cố định	Client linh hoạt	Nhị phân (protobuf)
JSON qua HTTP/1.1	1 endpoint, JSON	Tránh lấy dư dữ liệu, độ trễ thấp, streaming



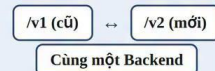
- Khi nào dùng REST: API công khai, CRUD, caching
- Khi nào dùng GraphQL: mobile app, query linh hoạt
- Khi nào dùng gRPC: microservices, streaming, độ trễ thấp

📦 Q16 Versioning API?

Đáp án: Cần đảm bảo tương thích ngược để tránh phá vỡ client hiện tại khi API thay đổi.

Các loại:

- URL versioning - `/v1/users` vs `/v2/users` (TỐT NHẤT)
- Header - Accept: `application/vnd.api.v1+json`
- Query param - `?version=1`



Thực hành tốt nhất: URL versioning phổ biến nhất, rõ ràng & dễ cache/cdn.

💡 Vì sao nhà tuyển dụng hỏi câu này:

- Kiểm tra tư duy thiết kế API cho thanh toán & khả năng mở rộng
- Cho thấy bạn xử lý được request trùng & dữ liệu lớn
- Hợp đồng API = xương sống của microservices

📖 Ví dụ thực tế:

- Stripe dùng Idempotency-Key cho các khoản charge
- Newsfeed Instagram dùng cursor pagination
- Netflix dùng GraphQL cho mobile
- gRPC dùng trong microservices nội bộ
- Google Maps API dùng `/v1` / `/v2` URL versioning

💬 API = Cách các service giao tiếp - thiết kế để an toàn, mở rộng & bền vững!



Q17-Q20 AUTH & BẢO MẬT

Q17 JWT vs Session vs OAuth2 là gì?

Đáp án:

- JWT = Stateless • header.payload.signature [Client → token trong box → Server xác thực]
- Session = Stateful • Server lưu SID [Client → Cookie → Memory/Redis → Server]
- OAuth2 = luồng Auth Server [User → App → Auth Server → Code → Token] • code grant flow
- Vì sao cần: đánh đổi giữa khả năng mở rộng và bảo mật
- Dùng ở đâu: JWT - microservices, Session - web app, OAuth2 - Google Login

Q18 JWT hoạt động thế nào?

Encode: `base64(header).base64(payload).signature`

Header + **Payload** + **Signature**

Login → Server tạo Access token (ngắn hạn 15 phút) + Refresh token (dài hạn 7 ngày, xoay vòng)

- Vì sao cần: xác thực stateless, giảm số lần truy vấn DB
- Dùng ở đâu: xác thực API, mobile app

Authentication vs Authorization?

Auth = Bạn là ai?

- đăng nhập, mật khẩu, sinh trắc học

Ví dụ:

- Authentication: đăng nhập bằng email/mật khẩu

Vì sao cần / Dùng ở đâu: RBAC, ABAC

Authz = Bạn được làm gì?

- quyền hạn, RBAC / vai trò

Ví dụ:

- Authorization: Admin được xóa, User chỉ được đọc

Q20 Cách bảo mật API?

- ☒ HTTPS
- ☒ Rate limiting
- ☒ JWT/OAuth2
- ☒ Input validation
- ☒ CORS
- ☒ API keys
- ☒ 3DS

Vì sao cần: ngăn tấn công (XSS, CSRF, DDoS)

Dùng ở đâu: API Gateway, Payment APIs.

Vì sao nhà tuyển dụng hỏi câu này:

- Kiểm tra nền tảng auth & kiến thức bảo mật token
- Cho thấy bạn hiểu stateless vs stateful
- Backend = tư duy bảo mật là ưu tiên hàng đầu

Ví dụ thực tế:

- JWT → xác thực microservices của Netflix
- Session → session web của Amazon dùng Redis
- OAuth2 → đăng nhập bằng Google / GitHub
- API Security → Stripe dùng HTTPS + Rate limit + API keys

Bạn đã nắm auth - nền tảng bảo mật Backend đã rõ ràng!

→ Swipe to Page 07



Q21-Q24

MẪU THIẾT KẾ MICROSERVICES

Q21: Monolith vs Microservices là gì?

Đáp án: Monolith = một codebase, một lần deploy.
Microservices = các service nhỏ độc lập, mỗi service có DB riêng.

Monolith App → **One deploy**

vs → decoupled →

Ưu • Nhược:

Monolith: Đơn giản, phát triển nhanh, dễ test | Khó mở rộng, một lỗi = sập toàn bộ

Microservices: Mở rộng tốt, deploy độc lập, đa dạng công nghệ | Phức tạp, độ trễ mạng, giao dịch phân tán

Vì sao cần: bắt đầu nhỏ vs mở rộng traffic khổng lồ

Dùng ở đâu: Monolith → startup giai đoạn đầu, Micro → Netflix, Amazon

Q22: Microservices giao tiếp thế nào?

Đáp án: 2 cách: Sync vs Async

Sync: REST (HTTP/JSON) - đơn giản, gRPC (protobuf) - nhanh.
Gọi trực tiếp, chờ phản hồi.

Async: Message Queue / Event Bus - Kafka, RabbitMQ. Bắn đi rồi thôi (fire & forget).

Service A → **REST/gRPC** →

Service B (sync, chờ phản hồi)

Service A → **Kafka Queue** →

Service B (async, fire & forget)

- Vì sao cần: giảm phụ thuộc, tăng khả năng phục hồi
- Dùng ở đâu: Sync → request của user, Async → đặt hàng xong → gửi email

Q23: Service Discovery là gì?

Đáp án: Cách một service tìm IP của service khác khi IP thay đổi (auto scaling).

Loại: Client-side Discovery: Client hỏi Registry → nhận IP.

Server-side: Router/Load Balancer hỏi Registry.

Công cụ: Eureka, Consul, K8s DNS/CoreDNS

Service A (đăng ký/heartbeat) → **Service Registry** →

Service B (gọi qua IP:Port)

- Vì sao cần: IP thay đổi động trên cloud, auto-scale
- Dùng ở đâu: K8s, Eureka trong Spring Cloud

Q24: API Gateway Pattern là gì?

Đáp án: Điểm vào duy nhất cho mọi client. Xử lý Auth, Rate Limiting, Routing, Aggregation, Logging.

Client App / Mobile

Auth • Rate Limit • Routing • Logging

Auth Service

Order Service

Payment Service

Vì sao cần: tập trung các mối quan tâm chung (cross-cutting)

Dùng ở đâu: Netflix Zuul, AWS API Gateway, Kong

Vì sao nhà tuyển dụng hỏi câu này:

- Kiểm tra bạn có hiểu đánh đổi monolith vs micro
- Kiểm tra kiến thức thiết kế hệ phân tán thực tế
- Microservices = bắt buộc phải biết với quy mô FAANG

Ví dụ thực tế:

- Monolith → Shopify khởi đầu là monolith
- Microservices → Netflix có 1000+ service
- Service Discovery → K8s dùng DNS
- API Gateway → Amazon API Gateway xử lý hàng triệu request

Giờ bạn đã làm chủ microservices - trái tim của backend có khả năng mở rộng!

→ Swipe to Page 08



Q25-Q28 MESSAGE QUEUES

Q25 Message Queue vs Event Stream?

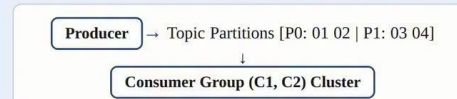
- Đáp án: Message Queue = point-to-point, message bị xóa sau khi tiêu thụ.
- Ví dụ: RabbitMQ, Amazon SQS.
- Event Stream = dựa trên log, lưu trữ lâu dài, có thể replay.
- Ví dụ: Kafka, Kinesis.



Vì sao cần: tách rời (decoupling), buffering, replay. Dùng ở đâu: xử lý đơn hàng, activity log.

Q26 Kafka hoạt động thế nào?

Đáp án: Log phân tán. Producer ghi vào Topic Partitions, Broker lưu trữ, Consumer đọc theo offset. Consumer Group để xử lý song song.



ZooKeeper / KRaft — Offset → con trỏ

- Vì sao cần: throughput cao, có thứ tự theo từng partition, mở rộng tốt, lưu bền
- Dùng ở đâu: newsfeed LinkedIn, sự kiện chuyển đi Uber

Q27 RabbitMQ vs Kafka?

Feature	RabbitMQ	Kafka
Loại	smart broker	log
Delivery	push	pull
Độ trễ	thấp (ms)	cao hơn
Throughput	trung bình	rất cao (M msg/giây)
Lưu trữ	in-memory + disk	disk log, tạm thời
Replay	không	có, lưu bền
Thứ tự	theo queue	theo partition

Khi nào dùng? RabbitMQ: tác vụ, RPC, job cần độ trễ thấp.
Kafka: streaming, phân tích, event sourcing.

Q28 Event-Driven Architecture là gì?

Đáp án: Các service giao tiếp qua event, không gọi trực tiếp. Publisher phát event, subscriber phản ứng bất đồng bộ.



- Lợi ích: tách rời, mở rộng tốt, phục hồi tốt, dễ mở rộng thêm tính năng
- Vì sao cần: giảm phụ thuộc chặt (loose coupling)
- Dùng ở đâu: luồng đặt hàng e-commerce, microservices

Vì sao nhà tuyển dụng hỏi câu này:

- Kiểm tra tư duy bất đồng bộ (async)
- Kiểm tra đánh đổi giữa decoupling & khả năng mở rộng
- Lựa chọn Kafka/RabbitMQ trong thực tế

Ví dụ thực tế:

- SQS → hàng đợi đơn hàng của Amazon
- Kafka → sự kiện xem video của Netflix 5M/giây
- RabbitMQ → hàng đợi task Celery cho email
- Event-Driven → chuyển Uber → ghép tài xế, thanh toán, thông báo qua event

Bạn đã chinh phục Message Queues - 4 mẫu thiết kế cho hệ thống bất đồng bộ!



Q29-Q32 SCALING & HẠ TẦNG

Q29 Docker vs Kubernetes là gì?

Đáp án: Docker đóng gói container cho app, K8s điều phối nhiều container.



Q30 Cách Scale Backend?

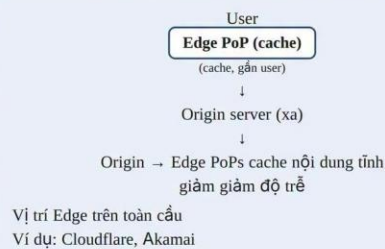
Vertical (dọc)
thêm CPU & RAM cho một server

Horizontal (ngang)
thêm nhiều server sau Load Balancer

Chiến lược:

- Load Balancer
- Sharding
- Caching
- CDN
- Async queues

Q31 CDN là gì?



Q32 WebSocket vs HTTP?

HTTP
request-response, ngắn hạn
half duplex

WebSocket
bền, hai chiều
full duplex

Handshake:

HTTP Upgrade → 101 Switching Protocols
Dùng ở đâu: chat, tỷ số trực tiếp, giao dịch, gaming

Vì sao nhà tuyển dụng hỏi câu này:

- Kiểm tra kiến thức về scale hạ tầng
- Cho thấy bạn hiểu hệ thống production
- Kiểm tra tư duy thiết kế cloud-native

Ví dụ thực tế:

- API Gateway rate limiting → 1000 req/phút mỗi user
- Cache-Aside → cache newsfeed của Instagram
- Load Balancer → AWS ELB dùng Round Robin
- CAP → Cassandra = AP, MongoDB = CP

Caching + Scaling = Hệ thống sống sót qua cơn bão traffic!

Swipe to Page 10 →

Q33-Q36

TRANSACTION & LOCK

Q33 Isolation Level của Transaction là gì?

Đáp án: Kiểm soát tính hiển thị của các transaction đang chạy đồng thời.

Danh sách: Read Uncommitted • Read Committed • Repeatable Read • Serializable (thấp → cao)

	Read Committed	Repeatable Read	Serializable
Dirty Read	✓	✗	✗
Non-Repeatable	✓	✗	✗
Phantom	✓	✓	✗

- Vì sao cần: ngăn dirty read, đảm bảo tính nhất quán, đánh đổi hiệu năng vs an toàn
- Dùng ở đâu: Banking • Inventory → Booking

Q34 Optimistic vs Pessimistic Locking?

Optimistic: không khóa → kiểm tra version → retry nếu không khớp

version = 1 T1 → Đọc A (v=1) → thành công

T2 → ghi v2 → thất bại → thử lại

Pessimistic: khóa dòng • SELECT FOR UPDATE

T1 → khóa dòng → T2 chờ. T1 → commit → mở khóa → T2 tiếp tục

- Vì sao cần: ngăn mất cập nhật (lost update), xử lý concurrency
- Dùng ở đâu: Optimistic → giỏ hàng e-commerce, Pessimistic → Đặt vé

Q35 Distributed Lock là gì?

Cần khóa xuyên nhiều server: Redis • Redlock • Zookeeper

S1 → S2 → Redis →

SETNX key (thất bại nếu đã có)

- Thuật toán: SET key value NX PX 5000 + Lua để release an toàn
- Vì sao cần: ngăn race condition giữa các instance, bảo vệ vùng tới hạn phân tán
- Dùng ở đâu: ngăn double-spend trong thanh toán, cron leader election → giảm tồn kho

Q36 Saga Pattern là gì?

Transaction phân tán xuyên microservices thông qua choreography hoặc orchestration kèm compensating transaction.

Order → Payment → Inventory (thành công)

nếu lỗi → bù trừ (compensate)

- Loại: Choreography → dùng event / Orchestration → dùng orchestrator
- Vì sao cần: tránh 2PC trong microservices, transaction chạy lâu dài
- Dùng ở đâu: luồng đặt hàng e-commerce → đặt vé du lịch • chuyển tiền

Vì sao nhà tuyển dụng hỏi câu này?

- Kiểm tra hiểu biết về concurrency trong DB
- Kiểm tra kiến thức hệ phân tán
- Backend phải xử lý được race condition

Ví dụ thực tế:

- Isolation → MySQL Repeatable Read vs Postgres Read Committed
- Optimistic Lock → version tồn kho của Amazon
- Distributed Lock → khóa thanh toán của Stripe
- Saga → saga đặt chuyến của Uber

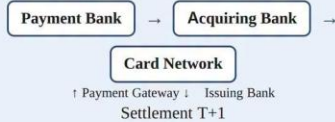
Bạn đã làm chủ transaction, lock & tính nhất quán phân tán - cực kỳ quan trọng cho backend FAANG!



Q37-Q40

THIẾT KẾ HỆ THỐNG THỰC TẾ

Q37 Hệ thống Payment hoạt động thế nào?



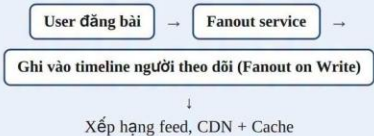
- Vì sao cần: ngăn tính phí trùng • xử lý lỗi • bảo mật
- Dùng ở đâu: Razorpay, Stripe, Amazon Pay.

Q38 UPI hoạt động thế nào?



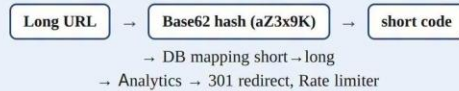
- Vì sao cần: chuyển tiền tức thì • VPA không cần lộ thông tin ngân hàng • hoạt động 24/7
- Dùng ở đâu: PhonePe, Google Pay, BHIM.

Q39 Thiết kế Newsfeed Instagram thế nào?



- Vì sao cần: feed nhanh → xếp hạng cá nhân hóa, mở rộng cho hàng triệu người dùng
- Dùng ở đâu: Instagram, Facebook, Twitter timeline.

Q40 Thiết kế URL Shortener thế nào?



- Vì sao cần: rút ngắn ký tự → theo dõi lượt click → hết hạn
- Dùng ở đâu: Bitly, TinyURL, Google Shortener.

Vì sao nhà tuyển dụng hỏi câu này:

- Kiểm tra tư duy thanh toán & newsfeed thực tế
- Cho thấy bạn hiểu dòng tiền, bảo mật, khả năng mở rộng
- Kiểm tra đánh đổi: idempotency, settlement, fanout

Ví dụ thực tế:

- Payment → Stripe dùng idempotency + webhook, Settlement T+1
- UPI → NPCI xử lý 10 tỷ+ giao dịch/tháng, dựa trên VPA
- Newsfeed Instagram → fanout tới 1 triệu follower, cache CDN
- URL Shortener → Bitly dùng Base62, 301 redirect + analytics

Giờ bạn đã biết thiết kế thực tế - thanh toán, UPI, feed & rút gọn URL!

Swipe to Page 12 →



Q41-Q44 CÁC MẪU LLD

Q41 Singleton vs Factory vs Builder - Khi nào dùng?

Singleton	Factory	Builder
Mục đích: một instance dùng chung toàn cục	Tạo object qua factory method	Xây dựng object phức tạp theo từng bước
Ví dụ: Logger / DB Connection	PaymentFactory.createPayment()	User.Builder() cho object phức tạp

- Vì sao cần: kiểm soát tạo object • linh hoạt + dễ đọc
- Dùng ở đâu: Logger, Factory cho payment, Builder cho config phức tạp.

Q42 Observer vs Pub-Sub - Khác nhau ra sao?

Observer (trong process)	Pub-Sub (qua broker)
Observer = đồng bộ trong bộ nhớ • Pub-Sub = tách rời qua broker Kafka/Redis	

- Vì sao cần: tách rời theo hướng sự kiện (event-driven decoupling)
- Dùng ở đâu: Observer → sự kiện UI, Pub-Sub → microservices.

Q43 Strategy Pattern - Ví dụ thực tế?

Đáp án: Các chiến lược thanh toán có thể hoán đổi cho nhau.

Context (PaymentContext) - has-a →

Strategy (PaymentStrategy interface)

implements

UPI Strategy

Card Strategy

NetBanking Strategy

context.setStrategy(new UPIStrategy()); context.pay();

- Vì sao cần: tránh if-else, tuân theo nguyên lý open/closed
- Dùng ở đâu: payment gateway, chiến lược sắp xếp (sorting).

Q44 Thiết kế LLD cho Rate Limiter thế nào?

```
<<interface>>
+ isAllowed(userId): boolean
```

```
TokenBucket implements RateLimiter
- tokens: int
- capacity: int
- refillRate: int
- lastRefill: long
+ consume(): boolean
```

UserBucket map: HashMap<userId, TokenBucket>

UserRequest

RateLimiter

TokenBucket

- Vì sao cần: ngăn lạm dụng, ngăn DDoS
- Dùng ở đâu: API Gateway, Auth.

Vì sao nhà tuyển dụng hỏi câu này:

- Kiểm tra nền tảng LLD + kiến thức design pattern
- Kiểm tra khả năng chọn đúng pattern cho đúng bài toán
- LLD thực sự = code mở rộng được, không chỉ chạy được.

Ví dụ thực tế:

- Singleton → Logger, ConfigManager trong production
- Pub-Sub → Kafka báo Order → Inventory → Email
- Strategy → Razorpay / PhonePe chuyển đổi UPI/Card/NetBanking
- RateLimiter → Token Bucket tại API Gateway 100 req/phút

... Làm chủ các mẫu LLD = thiết kế gọn gàng, mở rộng tốt cho phỏng vấn!

Swipe to Page 13 →



Q45-Q48 CÁC CASE STUDY LLD

Q45 SOLID Principles trực quan?

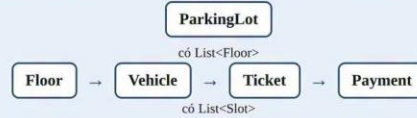
- S - Single Responsibility (mỗi class một nhiệm vụ)
- O - Open Closed (mở để mở rộng, đóng để sửa đổi)
- L - Liskov Substitution (lớp con thay thế được lớp cha)
- I - Interface Segregation (interface nhỏ, chuyên biệt)
- D - Dependency Inversion (phụ thuộc vào abstraction)



- Vì sao cần: OOP sạch, dễ bảo trì
- Thiết kế để mở rộng

P Q46 Thiết kế LLD cho Parking Lot?

Các class: Vehicle, Slot, Floor, ParkingLot, Ticket, Payment, Gate



- Vì sao cần: LLD được hỏi nhiều nhất về quan hệ OOP
- Dùng ở đâu: hệ thống Mall / Airport

Q47 Thiết kế Splitwise / Chia sẻ chi phí?

- Users, Groups, Expenses, Balance Sheet map<User, amount>
 - Đơn giản hóa bằng thuật toán Min Cash Flow / Heap
- A trả 1000 cho B, C, D → số dư: [A +750, B -250, C -250, D -250]
- tất toán với 2 giao dịch: B → A \$250, C → A \$250, D → A \$250
- Vì sao cần: đồ thị (graph) + tham lam (greedy)
 - Dùng ở đâu: Splitwise, SettleUp

Q48 Thiết kế hệ thống Notification?

Đa kênh: Email, SMS, Push, Template, dùng Strategy pattern để chọn kênh, dùng Observer cho subscriber



- Vì sao cần: khả năng mở rộng → tách rời (decoupled)
- Dùng ở đâu: thông báo của Swiggy, Amazon

Vì sao nhà tuyển dụng hỏi câu này:

- Kiểm tra SOLID + LLD
- Mô hình hóa OOP trong thực tế
- Cách dùng pattern Strategy, Observer

Ví dụ thực tế:

- Parking Lot → bãi đỗ xe của Uber
- Splitwise → 50 triệu người dùng
- Notification → push notification của Instagram
- SOLID → mọi công ty FAANG

Bạn đã làm chủ các case study LLD - vượt qua mọi vòng phỏng vấn thiết kế OOP!

Swipe to Page 14 →



Q49-Q50

QUY MÔ LỚN & LƯỒNG TRÌNH DUYỆT

... Q49 Cách xử lý 1 triệu RPS?

Đáp án: Mở rộng theo nhiều lớp

- CDN → cache tài nguyên tĩnh ở edge
- Load Balancer (L7) → phân phối traffic
- Caching: Redis Cluster hit 90%
- Horizontal scaling: tự động scale 100+ app server
- DB Read Replicas + Sharding (theo user_id)
- Async Queues: Kafka cho các thao tác ghi
- Rate Limiting: token bucket tại gateway



- Vì sao cần: ngăn sập hệ thống • xử lý traffic tăng đột biến → độ trễ thấp
- Dùng ở đâu: Big Billion Day của Flipkart • newsfeed Instagram → payment gateway

🌐 Q50 Chuyện gì xảy ra khi bạn gõ google.com? (Luồng trình duyệt)

Đáp án: Hành trình đầy đủ gồm 10 bước:

1. Kiểm tra cache trình duyệt
2. Tra cứu DNS → IP (142.250.x.x)
3. Bắt tay TCP: SYN/SYN-ACK/ACK
4. Bắt tay TLS → HTTPS bảo mật
5. Gửi HTTP Request GET google.com
6. Load Balancer → Định tuyến tới Data Center gần nhất
7. App Server + kiểm tra Cache (Redis)
8. Truy vấn DB nếu cache miss
9. Phản hồi → HTML/CSS/JS
10. Trình duyệt Render + paint (vẽ trang)



💡 Vì sao nhà tuyển dụng hỏi câu này:

- Kiểm tra tư duy quy mô lớn vượt xa lý thuyết
- Kiểm tra bạn có hiểu luồng end-to-end, không chỉ code
- 1M RPS + luồng trình duyệt = bài test độ chín trong system design

🔗 Ví dụ thực tế:

- 1M RPS → Netflix dùng CDN + LB + Redis + Cassandra sharding + Kafka
- google.com → Google dùng Anycast DNS + Global LB + Edge Cache
- Rate limit + replica → Amazon xử lý traffic flash sale
- DNS đến Render → cách Chrome tải instagram.com trong <1 giây

💬 **Bạn đã làm chủ high scale - gần đến lúc xem cheatsheet rồi!**

Swipe to Page 15 →



CHEATSHEET TỔNG KẾT & MẸO PHÒNG VẤN

TOP 50 CÂU HỎI PHÒNG VẤN BACKEND - TỔNG KẾT CUỐI 🚀

✅ Checklist phân loại 50 câu hỏi

📁 System Design (10 câu)

- Q1 Rate Limiting • Q2 Chiến lược Caching • Q3 Load Balancer Algos • Q4 CAP Theorem • Q5 Consistent Hashing
- Q6 CDN • Q7 Idempotency • Q8 Sharding vs Partitioning
- Q9 Eventual Consistency
- Q10 Distributed Lock

📁 DB (10 câu)

- Q11 Indexing • Q12 ACID vs BASE • Q13 SQL vs NoSQL • Q14 N+1 Query • Q15 Replication vs Sharding • Q16 Normalization
- Q17 Transaction Isolation • Q18 Optimistic vs Pessimistic
- Q19 DB Connection Pool
- Q20 Query Optimization

🔒 API Auth (10 câu)

- Q21 REST vs gRPC • Q22 JWT • Q23 OAuth2 • Q24 API Rate Limit Implementation • Q25 Pagination • Q26 API Versioning • Q27 Webhooks vs Polling
- Q28 Idempotency Key • Q29 HMAC Auth • Q30 API Gateway Patterns

📁 Microservices Queue (10 câu)

- Q31 Monolith vs Microservices • Q32 SAGA Pattern • Q33 Message Queue Kafka vs RabbitMQ • Circuit Breaker • Q35 Service Discovery
- Q36 Event Driven Architecture • Q37 Distributed Tracing • Q38 Delivery • Q39 Dead Letter Queue • Q40 Backpressure

🌟 LLD Thực tế (10 câu)

- Q41 Design TinyURL • Q42 Design Instagram • Q43 Design Payment System • Q44 LLD Parking Lot • Q45 Design Rate Limiter LLD • Q46 OOP SOLID
- Q47 Design Cache Library • Q48 Concurrency Handling • Q49 Design Notification System • Q50 Design Chat System

+ Top 5 lỗi cần tránh

- ❌ Nhảy vào code ngay mà không làm rõ yêu cầu
- ❌ Bỏ qua các đánh đổi (trade-off)
- ❌ Không xử lý trường hợp lỗi
- ❌ Thiết kế quá phức tạp
- ❌ Quên giám sát & khả năng mở rộng

🔍 Chiến lược phỏng vấn - 5 bước

1. Làm rõ yêu cầu -

Hỏi về yêu cầu, quy mô, ràng buộc

2. Thiết kế tổng quan (High Level) -

Client → Gateway → LB → Services →

DB • Cache • Queue

3. Đào sâu (Deep Dive) -

Chọn 2-3 thành phần để phân tích sâu

4. Đánh đổi (Tradeoffs) -

Thảo luận ưu/nhược điểm, CAP, độ trễ vs tính nhất quán

5. Theo dõi (Follow up) -

🚩 Giám sát, mở rộng, xử lý lỗi

📖 Top 3 sách nên đọc

- 📖 1. Designing Data-Intensive Applications
- 📖 2. System Design Interview - Alex Xu
- 📖 3. Clean Code + Clean Architecture

💬 Chúc bạn tự tin bước vào vòng phỏng vấn Backend tiếp theo! 🚀 ✨